

## Cell 4: FINAL CLEAN RDF TRIPLE EXTRACTION

This is where your **Graph RAG** actually gets intelligence

I'll explain it **very clearly step-by-step (no confusion)**

---

### What This Code Does (Big Picture)

Raw Text → Clean → Extract → RDF Triples → (Subject, Relation, Object)

👉 Example output:

('Combustion', 'uses', 'Air')

('Fuel System', 'supplies', 'Fuel')

👉 This is what builds your **knowledge graph**

---

### 1. `clean_entity()` → Clean Subject

`def clean_entity(text):`

👉 Purpose:

Clean and standardize entity names (nodes)

---

**What it does:**

#### Step 1: Remove punctuation

```
text = re.sub(r'^\w\s', "", text)
```

👉 Example:

"Fuel System." → "Fuel System"

---

#### Step 2: Remove stop words

```
stop_words = {"the", "a", "an", "and", "or", "of", "to"}
```

👉 Example:

"The Fuel System" → "Fuel System"

---

#### Step 3: Convert to clean format

```
return " ".join(words).title()
```

👉 Output:

"fuel system" → "Fuel System"

---

## 🔥 2. `normalize_relation()` → Fix Relations

```
def normalize_relation(rel, obj):
```

👉 Purpose:

Fix relation names to proper format

---

**Example:**

"consists of" → "consists\_of"

👉 Why?

Graph needs clean relation labels

---

## 🔥 3. `clean_object()` → Clean Object

```
def clean_object(text):
```

👉 Purpose:

Clean the object part (right side of triple)

---

**What it does:**

**Remove punctuation**

"Air." → "Air"

---

**Keep only first 3 words**

```
return " ".join(words[:3])
```

👉 Example:

"Fuel through injectors into chamber"  
→ "Fuel Through Injectors"

👉 Why?

Long text = bad graph ❌

Short clean node = good graph ✅

---

#### 🔥 4. `extract_triples_rdf_final()` → MAIN FUNCTION

👉 This is the **core logic**

---

##### ● Step 1: Initialize

```
triples = []  
seen = set()
```

👉 Purpose:

Store triples  
Avoid duplicates

---

##### ● Step 2: Define Valid Relations

```
valid_relations = [...]
```

👉 Only extract meaningful actions like:

uses, produces, supplies, drives

👉 Avoid noise ❌

---

##### ● Step 3: Remove Weak Subjects

```
invalid_subjects = {...}
```

👉 Remove bad nodes:

"and", "system", "components" ❌

---

##### ● Step 4: Loop Documents

```
for source, text in documents.items():
```

👉 Process each PDF

---

### ● Step 5: Split into Lines

```
lines = text.split("\n")
```

👉 Work line-by-line

---

### ● Step 6: Filter Small Lines

```
if len(line) < 10:  
    continue
```

👉 Skip useless text

---

### ● Step 7: Find Relations

```
if rel in line.lower():
```

👉 Example:

```
"Fuel system supplies fuel"  
→ match "supplies"
```

---

### ● Step 8: Split Sentence

```
parts = re.split(rel, line)
```

👉 Example:

```
"Fuel system supplies fuel"  
  
→ ["Fuel system", "fuel"]
```

---

### ● Step 9: Extract SUBJECT

```
subject_words = parts[0].split()  
subject = last 2 words
```

👉 Example:

```
"Fuel system" → "Fuel System"
```

---

### ● Step 10: Clean Subject

```
subject = clean_entity(subject)
```

---

### ● Step 11: Remove Invalid Subject

```
if subject.lower() in invalid_subjects:  
    continue
```

---

### ● Step 12: Extract OBJECT

```
obj_raw = parts[1]  
obj = clean_object(obj_raw)
```

---

### ● Step 13: Fix Relation

```
rel_fixed = normalize_relation(rel, obj_raw)
```

---

### ● Step 14: Create Triple

```
triple = (subject, rel_fixed, obj, source)
```

👉 Example:

```
('Fuel System', 'supplies', 'Fuel', 'engine.pdf')
```

---

### ● Step 15: Remove Duplicates

```
if triple not in seen:
```

---

### ● Step 16: Store Triple

```
triples.append(triple)
```

---

### ● Step 17: Return Output

```
return triples
```

---

## 🔥 Final Output

```
triples = extract_triples_rdf_final(documents)
```

👉 Example:

```
[  
  ('Combustion', 'uses', 'Air'),  
  ('Fuel System', 'supplies', 'Fuel'),  
  ('Compressor', 'compresses', 'Air')  
]
```

---

## 🧠 Why This is MOST IMPORTANT

Because:

Triples = Graph Foundation

---

## 🔥 Pipeline Connection

PDF

↓

Text

↓

THIS CODE 🔥

↓

Triples

↓

Graph

↓

Retriever

↓

LLM

---

## 🎯 One-Line Summary

This code extracts RDF-style triples (subject, relation, object) from raw document text by identifying relationships, cleaning entities, and structuring the data for building a knowledge graph.

---

 **Real Insight (Very Important)**

Better triples → Better graph → Better answers

---

 **You Understood This = You Understand Graph RAG Core**

---

 **Next Step**

Send next cell:

 **Graph building (NetworkX)**

Now we will convert this into **actual graph** 🔥